

Android Static Application Security Testing

A practical vulnerability reference for source and bytecode review — what the vulnerability is, why it happens, and exactly how to find it before you ever launch an emulator.

Table of Contents

Every entry follows the same four-part structure: the vulnerability, why it happens, how to find it statically, and its real-world impact — so you can scan this manual during a live review without hunting for the logic.

01 · METHODOLOGY

— How to Use This Manual	4
--------------------------	---

02 · DATA STORAGE & PRIVACY — MASVS-STORAGE

01 Hardcoded Secrets & API Keys	5
02 Insecure Local Data Storage	5
03 Insecure Logging of Sensitive Data	6
04 android:allowBackup Misconfiguration	6
05 Tapjacking / Missing FLAG_SECURE	7

03 · CRYPTOGRAPHY — MASVS-CRYPTO

06 Weak or Broken Cryptography	7
07 Insecure Random Number Generation	8

04 · NETWORK COMMUNICATION — MASVS-NETWORK

08 Cleartext Traffic & Weak Network Security Config	8
09 Missing or Broken Certificate Pinning	9

05 · PLATFORM INTERACTION — MASVS-PLATFORM

10 Exported Components Without Permission	9
11 Insecure WebView Configuration	10
12 SQL Injection via Content Providers	10
13 Path Traversal in File Handling	11
14 Insecure Deep Link / Intent Handling	11
15 PendingIntent Mutability Issues	12

06 · CODE QUALITY & BUILD CONFIG — MASVS-CODE

16 android:debuggable Left Enabled	12
17 Vulnerable Third-Party Dependencies (SCA)	13
18 Firebase / Cloud Endpoint Misconfiguration	13

07 · APPENDIX

— One-Shot Static Recon Script	14
— Severity & Tag Legend	14

How to Use This Manual

Static application security testing (SAST) on Android means reviewing the APK without running it — the manifest, decompiled source, resources, and bundled native libraries. It always comes before dynamic testing: ten minutes here tells you exactly where a Frida session should focus.

01

Decompile

Unpack the APK with apktool for resources/manifest and jadx for readable Java source. Never review raw smali unless jadx fails on a method.

02

Triage the Manifest

Flags, exported components, and permissions live here. This single file usually points to 60% of your findings before you open a source file.

03

Pattern-Match Source

Grep for the vulnerable API signatures in this manual across the jadx output. Confirm each hit is reachable — a string match alone isn't a finding.

04

Write the Finding

Root cause → impact → fix → alternative. A grep hit becomes a finding only once you can state the business impact in one sentence.

Rule of thumb: a static finding is "confirmed" when you can point to the exact file and line, explain why it's reachable by an attacker with just the APK, and state what breaks as a result. Anything short of that is a lead, not a finding — keep it in your notes, not your report.

MASVS-XXXX

MASVS category tag

CRITICAL

Direct compromise, no auth needed

HIGH

Significant data/logic exposure

MEDIUM

Requires chaining or conditions

01 Hardcoded Secrets & API Keys

STORAGE

CRITICAL

THE VULNERABILITY

API keys, backend credentials, encryption keys, or third-party service secrets embedded directly as string literals in source, resources, or native libraries. Anyone with the APK has the secret.

ROOT CAUSE

Convenience during development — the key is never moved to a remote config or backend proxy before release, or a build script fails to strip debug credentials from the release variant.

HOW TO FIND IT

```
# Automated secrets/endpoint scan
apkleaks -f target.apk -o secrets_report.txt

# Manual grep across decompiled source
grep -rniE "api[_-]?key|secret|token|password|AIza[0-9A-Za-z_-]{35}|BEGIN (RSA|EC|PRIVATE) KEY"
jadx_src/

# Don't forget native libraries — secrets hide in .so files too
strings lib/arm64-v8a/*.so | grep -iE "key|secret|token"
```

```
private static final String API_KEY = "AIzaSyD4x9F...redacted...";
```

IMPACT → Full API abuse under the app's identity, billing fraud, or direct backend compromise if the key is scoped broadly (e.g. an unrestricted Firebase or cloud key).

02 Insecure Local Data Storage

STORAGE

HIGH

THE VULNERABILITY

Sensitive data (tokens, PII, PINs) written to plaintext SharedPreferences, unencrypted SQLite databases, or world-readable internal/external files instead of an encrypted store.

ROOT CAUSE

Devs reach for the simplest storage API (SharedPreferences) without realizing app-private ≠ encrypted — it's still plaintext on disk, extractable without root.

HOW TO FIND IT

```
# Find storage writes in decompiled source
grep -rn "getSharedPreferences(\|openFileOutput(\|MODE_WORLD" jadx_src/

# Confirm on-device: pull and read without root
adb backup -f data.ab -noapk com.app.target
adb shell run-as com.app.target cat shared_prefs/user_prefs.xml
```

```
SharedPreferences prefs = getSharedPreferences("auth", MODE_PRIVATE);
prefs.edit().putString("session_token", token).apply(); // plaintext on disk
```

IMPACT → Session tokens/PII recovered via adb backup, on any rooted device, or in a lost/resold phone — no exploit chain required, just file access.

03 Insecure Logging of Sensitive Data

STORAGE

MEDIUM

THE VULNERABILITY

Tokens, passwords, or PII written to Logcat via `Log.d/e/i/v/w`. Captured by crash reporters, bug-report exports, and any logcat-reading tool on a rooted or debuggable device.

ROOT CAUSE

Debug logging left in the release build — devs log full request/response payloads while building an auth flow and forget to strip it before shipping.

HOW TO FIND IT

```
# Grep log calls near sensitive variable names
grep -rniE "Log\.(d|e|i|v|w)\(.*(token|password|pin|ssn|auth|secret)" jadx_src/

# Live confirmation on a debuggable build
adb logcat | grep -i "token|password"
```

```
Log.d("AuthManager", "Login success, token: " + authToken);
```

IMPACT → Token/PII exposure via Logcat, bundled crash-reporting SDKs (Crashlytics, Sentry), or a support-generated bug report shared outside the security team.

04 android:allowBackup Misconfiguration

STORAGE

HIGH

THE VULNERABILITY

`android:allowBackup="true"` (the default if unset) with no restrictive `dataExtractionRules` / `fullBackupContent` lets the entire app sandbox be pulled via `adb backup`.

ROOT CAUSE

Nobody touches the flag, so it silently defaults to enabled. Teams assume "no root needed" isn't a real threat model — it is: USB debugging + unlocked device is enough.

HOW TO FIND IT

```
# Manifest check
grep "android:allowBackup" AndroidManifest.xml

# Android 12+ granular rules — check these too, allowBackup alone isn't the full picture
cat res/xml/data_extraction_rules.xml
cat res/xml/backup_rules.xml

# Exploit locally
adb backup -f loot.ab -noapk com.app.target
java -jar abe.jar unpack loot.ab loot.tar
```

```
<application android:allowBackup="true" ... > <!-- no dataExtractionRules -->
```

IMPACT → Full app data extraction — databases, `SharedPreferences`, cached auth tokens — from any unlocked device with USB debugging on, no root required.

05 Tapjacking / Missing FLAG_SECURE

STORAGE

MEDIUM

THE VULNERABILITY

Sensitive screens (PIN entry, payment, seed phrase) missing FLAG_SECURE can be screenshotted/screen-recorded by other apps, and overlay-based tapjacking can trick users into confirming actions.

ROOT CAUSE

FLAG_SECURE is opt-in per-Activity, not global — it's easy to add it to the login screen and forget it on a later-added payment or MFA-setup screen.

HOW TO FIND IT

```
# List sensitive activities, then confirm FLAG_SECURE is set on each
grep -rn "FLAG_SECURE" jadx_src/

# Cross-reference against every Activity handling PIN/payment/OTP
grep -rln "class.*Activity" jadx_src/ | xargs grep -Li "FLAG_SECURE"
```

```
public class PinEntryActivity extends AppCompatActivity {
    // no getWindow().setFlags(FLAG_SECURE, FLAG_SECURE) anywhere in onCreate()
}
```

IMPACT → PIN/OTP capture via screen recording malware or accessibility-service overlays; recent-apps screenshot leaks sensitive UI state to other apps.

06 Weak or Broken Cryptography

CRYPTO

CRITICAL

THE VULNERABILITY

Deprecated algorithms (DES, RC4, MD5/SHA1 for password hashing), ECB mode (pattern-leaking, no IV), or a static hardcoded key/IV baked into the APK.

ROOT CAUSE

Copy-pasted crypto snippets from Stack Overflow, or a key that "has to live somewhere" and gets embedded in code instead of derived from the Android Keystore.

HOW TO FIND IT

```
# Weak algorithm / mode signatures
grep -rniE "Cipher.getInstance\\(\"(DES|RC4|AES|ECB|Blowfish)\" jadx_src/
grep -rniE "MessageDigest.getInstance\\(\"(MD5|SHA-?1)\\)\" jadx_src/

# Hardcoded key/IV byte arrays
grep -rn "byte\\[\\] key = {\\|byte\\[\\] iv = {" jadx_src/
```

```
Cipher c = Cipher.getInstance("AES/ECB/PKCS5Padding");
byte[] key = {0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08}; // static, extractable
```

IMPACT → Any attacker with the APK derives the key and decrypts every user's stored data; ECB mode leaks structural patterns even without breaking the key.

07 Insecure Random Number Generation

CRYPTO

HIGH

THE VULNERABILITY

`java.util.Random` (a seeded, predictable PRNG) used to generate OTPs, session tokens, password-reset codes, or crypto nonces instead of `SecureRandom`.

ROOT CAUSE

`Random` is the default autocomplete suggestion in most IDEs; its non-cryptographic nature isn't obvious from the API name alone.

HOW TO FIND IT

```
# Flag Random usage, then check if output feeds a security-relevant value
grep -rn "new Random(" jadx_src/ | grep -v "SecureRandom"

# Confirm it's not just used for UI/animation jitter
grep -B3 -A3 "new Random(" jadx_src/**/OtpGenerator.java
```

```
int otp = new Random().nextInt(1000000); // seeded by time – predictable
```

IMPACT → Predictable OTPs/tokens brute-forced or reconstructed from the seed window, enabling account takeover or session hijacking.

08 Cleartext Traffic & Weak Network Security Config

NETWORK

HIGH

THE VULNERABILITY

HTTP endpoints hardcoded in source, or `network_security_config.xml` setting `cleartextTrafficPermitted="true"` (base config or per-domain).

ROOT CAUSE

A staging/legacy endpoint never migrated to HTTPS, or a broad cleartext override added to debug a proxy and never scoped back down for release.

HOW TO FIND IT

```
# Manifest flag
grep "android:usesCleartextTraffic" AndroidManifest.xml

# Network security config
cat res/xml/network_security_config.xml | grep -i "cleartext"

# Hardcoded http:// endpoints in source
grep -rn '"http://' jadx_src/ | grep -v "schemas.android.com\|w3.org"
```

```
<base-config cleartextTrafficPermitted="true">
  <trust-anchors><certificates src="system"/></trust-anchors>
</base-config>
```

IMPACT → Credential/session interception on any hostile network (public Wi-Fi, malicious AP) — no certificate manipulation needed at all.

09 Missing or Broken Certificate Pinning

NETWORK

HIGH

THE VULNERABILITY

No pinning at all on a high-value app, or a custom `X509TrustManager` with an empty/no-op `checkServerTrusted()` that silently accepts every certificate.

ROOT CAUSE

The empty-body `TrustManager` pattern gets copy-pasted to "fix" an SSL error during dev against a self-signed proxy, then ships unchanged to production.

HOW TO FIND IT

```
# Presence check – is pinning implemented at all?
grep -rn "CertificatePinner" jadx_src/

# The dangerous pattern – custom trust manager that trusts everything
grep -rn -A3 "implements X509TrustManager" jadx_src/
```

```
public void checkServerTrusted(X509Certificate[] chain, String authType) {
    // empty – accepts any certificate presented, including an attacker's
}
```

IMPACT → Complete MITM on TLS traffic even with a valid-looking handshake — attacker-controlled proxy reads and modifies every request/response.

10 Exported Components Without Permission

PLATFORM

CRITICAL

THE VULNERABILITY

An Activity, Service, Receiver, or Provider with `android:exported="true"` and no matching `android:permission` — any installed app can launch or query it directly.

ROOT CAUSE

Pre-API 31, any component with an `<intent-filter>` defaults to exported automatically — devs add a filter for deep linking and don't realize they just opened IPC access too.

HOW TO FIND IT

```
# List every exported component with no permission guard
grep -B5 'exported="true"' AndroidManifest.xml | grep -L "android:permission"

# Exploit confirmation via Drozer
dz> run app.activity.info -a com.app.target -u
dz> run app.activity.start --component com.app.target .AdminPanelActivity
```

```
<activity android:name=".AdminPanelActivity" android:exported="true" />
<!-- no android:permission attribute -->
```

IMPACT → Any installed app launches privileged screens or bypasses the intended auth flow entirely — one of the most common real-world Android CVE classes.

11 Insecure WebView Configuration

PLATFORM

CRITICAL

THE VULNERABILITY

`addJavaScriptInterface()` exposing native methods to loaded JS, combined with `setJavaScriptEnabled(true)` + `setAllowFileAccess(true)`, or SSL errors silently ignored via `proceed()`.

ROOT CAUSE

A hybrid app needs JS-to-native calls for a feature (payments, native camera access) and exposes far more surface than that one feature requires.

HOW TO FIND IT

```
# JS bridge exposure
grep -rn "addJavaScriptInterface" jadx_src/

# Dangerous file-access combo
grep -rn "setAllowFileAccess(true)|setAllowUniversalAccessFromFileURLs(true)" jadx_src/

# SSL errors being swallowed
grep -rn -A2 "onReceivedSslError" jadx_src/ | grep "proceed()"
```

```
webView.addJavaScriptInterface(new JSBridge(), "Android"); // exposes native methods to any
loaded page
```

IMPACT → JS-to-native bridge abuse leading to arbitrary local file read/write or native method invocation if any attacker-controlled content ever loads in the WebView.

12 SQL Injection via Content Providers

PLATFORM

HIGH

THE VULNERABILITY

Raw string concatenation building SQL queries (`rawQuery`, `execSQL`) instead of parameterized ? placeholders — especially inside an exported `ContentProvider.query()`.

ROOT CAUSE

String concatenation is the path of least resistance; the `selection/selectionArgs` split in the `ContentProvider` API is easy to bypass by building the `WHERE` clause manually.

HOW TO FIND IT

```
# Concatenation into query methods
grep -rn "rawQuery(\|execSQL(" jadx_src/ | grep "+"

# Review the provider's query() for unsanitized pass-through
grep -A15 "public Cursor query" jadx_src/**/*Provider.java
```

```
db.rawQuery("SELECT * FROM users WHERE name='" + input + "'", null);
```

IMPACT → Local database exfiltration by any app that can query the exported provider — auth tables, cached PII, session data.

13 Path Traversal in File Handling

PLATFORM

HIGH

THE VULNERABILITY

A file path built from attacker-controlled input (Intent extra, deep link param) with no traversal sanitization, or an overly broad FileProvider path config using `<external-path path="."/>`.

ROOT CAUSE

Devs trust the filename param because "it's our own app calling it" — forgetting the component may be exported and reachable from any app.

HOW TO FIND IT

```
# File() built from external input
grep -rn "new File(" jadx_src/ | grep "getStringExtra\|getIntent()"

# Overly broad FileProvider path config
cat res/xml/file_paths.xml | grep 'path="."'
```

```
File f = new File(baseDir, intent.getStringExtra("filename"));
// filename = "../../../databases/auth.db" — no traversal check
```

IMPACT → Arbitrary app-private file read/write via an exported component, chained to pull auth databases or overwrite app config.

14 Insecure Deep Link / Intent Handling

PLATFORM

HIGH

THE VULNERABILITY

A deep link (custom scheme or App Link) whose parameters are trusted directly — used to auto-login, load a WebView URL, or make an authorization decision without server-side re-validation.

ROOT CAUSE

Deep links are treated as "internal" navigation events, but they're really untrusted external input — any app or a phishing link can fire one.

HOW TO FIND IT

```
# Find declared schemes
grep -A5 "android:scheme" AndroidManifest.xml

# Trace how the handling Activity uses intent data
grep -rn "getIntent().getData()" jadx_src/
```

```
Uri data = getIntent().getData();
String authToken = data.getQueryParameter("token");
loginUser(authToken); // trusted with no server-side check
```

IMPACT → Account takeover via a crafted link delivered over SMS/email/QR — the classic deep-link auth-bypass pattern seen across multiple public disclosures.

15 PendingIntent Mutability Issues

PLATFORM

MEDIUM

THE VULNERABILITY

A `PendingIntent` created without `FLAG_IMMUTABLE` (mutable by default pre-targetSdk 31) lets another app fill in unset Intent fields — action, data, extras — before it fires.

ROOT CAUSE

Older code predates Android 12's immutability requirement and was never revisited when the app raised its target SDK.

HOW TO FIND IT

```
# Find PendingIntent creation missing FLAG_IMMUTABLE
grep -rn "PendingIntent.get(Activity|Broadcast|Service)" jadx_src/ | grep -v "FLAG_IMMUTABLE"
```

```
PendingIntent.getBroadcast(ctx, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT);
// missing FLAG_IMMUTABLE – hijackable by another app on the device
```

IMPACT → Malicious app injects its own action/extras into the `PendingIntent`, hijacking the operation it was meant to perform (notification actions are a common target).

16 android:debuggable Left Enabled

CODE

CRITICAL

THE VULNERABILITY

android:debuggable="true" present in the release build's merged manifest — allows a JDWP debugger to attach to a production install and read/modify runtime state.

ROOT CAUSE

A build-flavor misconfiguration where the debug flag isn't explicitly forced off in the release variant, or a CI pipeline signs a debug-flavor build with the release key by mistake.

HOW TO FIND IT

```
# Check merged manifest of the release APK specifically
grep "android:debuggable" release_decoded/AndroidManifest.xml

# Confirm live on an installed release build
adb shell run-as com.app.target id
# succeeding here without root = debuggable is on
```

```
<application android:debuggable="true" ...> <!-- shipped in a signed release build -->
```

IMPACT → Full runtime introspection and memory access on a production install — JDWP attach, live method hooking, and credential extraction without root.

17 Vulnerable Third-Party Dependencies (SCA)

CODE

HIGH

THE VULNERABILITY

A bundled SDK (networking, analytics, ad, JSON parsing) with a known CVE — deserialization bugs, SSRF, or RCE inherited into the app with zero app-code changes required to trigger it.

ROOT CAUSE

Dependencies are pinned once at project start and never revisited; nobody owns the "keep third-party libs current" responsibility.

HOW TO FIND IT

```
# If you have source access
./gradlew dependencies
dependency-check --project target --scan build.gradle

# APK-only: fingerprint bundled library versions from package/class names
grep -rn "okhttp3\|com.google.gson\|com.squareup" jadx_src/ | grep -i version
```

```
implementation 'com.squareup.okhttp3:okhttp:3.12.0' // multiple known CVEs in this range
```

IMPACT → Inherited vulnerability class (RCE, SSRF, DoS) that exists purely because of an outdated dependency — no custom app logic needs to be flawed at all.

18 Firebase / Cloud Endpoint Misconfiguration

[CODE](#)

CRITICAL

THE VULNERABILITY

A hardcoded Firebase Realtime Database / Storage URL, or AWS S3 bucket reference, pointing at a backend with open (unauthenticated) read/write security rules.

ROOT CAUSE

Default Firebase rules are permissive during development and never locked down before the project URL ships inside a public APK.

HOW TO FIND IT

```
# Extract cloud endpoint strings from the APK
grep -rniE "firebaseio\.com|firebasestorage\.googleapis\.com|amazonaws\.com" jadx_src/

# Test the discovered endpoint for unauthenticated access
curl https://PROJECT-ID.firebaseio.com/.json
```

```
private static final String DB_URL = "https://myapp-prod.firebaseio.com";
// rules.json: { ".read": true, ".write": true }
```

IMPACT → Full unauthenticated database dump — user records, PII, sometimes auth tables — one of the highest-yield findings in mobile bug bounty programs.

Appendix · One-Shot Static Recon Script

A single triage pass across the highest-signal checks in this manual. Run this first on any new APK — it won't replace manual review, but it tells you within a minute which of the 18 classes above deserve a closer look.

```
#!/bin/bash
# android_static_triage.sh — run after: apktool d + jadx -d
SRC=jadx_src/
MANIFEST=decoded/AndroidManifest.xml

echo "[01] Hardcoded secrets"
grep -rniE "api[_-]?key|secret|password|BEGIN (RSA|EC) PRIVATE" $SRC

echo "[04] allowBackup"
grep "android:allowBackup" $MANIFEST

echo "[06] Weak crypto"
grep -rniE "AES|ECB|DES|RC4|MD5|SHA-1" $SRC

echo "[08] Cleartext traffic"
grep "usesCleartextTraffic" $MANIFEST

echo "[10] Exported components without permission"
grep -B5 'exported="true"' $MANIFEST | grep -L "permission"

echo "[11] WebView JS bridge"
grep -rn "addJavaScriptInterface" $SRC

echo "[16] Debuggable flag"
grep "android:debuggable" $MANIFEST

echo "[18] Cloud endpoints"
grep -rniE "firebaseio\.com|amazonaws\.com" $SRC
```

Portfolio idea: extend this script into a proper Python tool — wrap apktool + jadx + these greps into one automated pipeline with a JSON/Markdown report output. It's the same shape as your Argus and XXE-Hunter projects, and it's a strong standalone GitHub piece for a Product Security portfolio.

VULNERABILITY	MASVS TAG	PRIMARY SIGNAL
01 Hardcoded Secrets	STORAGE	apkleaks / grep on key patterns
02 Insecure Local Storage	STORAGE	getSharedPreferences, MODE_WORLD_*
03 Insecure Logging	STORAGE	Log.d/e near sensitive vars
04 allowBackup	STORAGE	Manifest flag + backup rules XML
05 Tapjacking	STORAGE	Missing FLAG_SECURE on sensitive Activities
06 Weak Cryptography	CRYPTO	ECB / DES / RC4 / MD5 signatures
07 Insecure Random	CRYPTO	new Random() feeding OTP/token
08 Cleartext Traffic	NETWORK	usesCleartextTraffic, http:// literals

09 Broken Cert Pinning	NETWORK	Empty checkServerTrusted()
10 Exported Components	PLATFORM	exported=true, no permission
11 Insecure WebView	PLATFORM	addJavascriptInterface
12 SQLi via Provider	PLATFORM	rawQuery/execSQL string concat
13 Path Traversal	PLATFORM	new File() from Intent extras
14 Insecure Deep Links	PLATFORM	getIntent().getData() trusted directly
15 PendingIntent Mutability	PLATFORM	Missing FLAG_IMMUTABLE
16 debuggable Enabled	CODE	Manifest flag on release build
17 Vulnerable Dependencies	CODE	Gradle SCA / bundled SDK versions
18 Firebase Misconfiguration	CODE	Open firebaseio.com/.json